

RespiraCheck — Frontend & Integration Documentation

Table of Contents

1. Project Overview & Architecture
 2. Technology Stack
 3. File Structure
 4. Frontend Entry Point — index.html
 5. React Entry Point — main.jsx
 6. Vite Configuration — vite.config.js
 7. Main Application Component — App.jsx
 8. Stylesheet — style.css
 9. Backend API — app.py
 10. Grad-CAM Module — gradcam.py
 11. Frontend-Backend Integration Flow
 12. API Endpoint Reference
 13. Data Flow Diagrams
-

1. Project Overview & Architecture

RespiraCheck is an AI-powered chest X-ray pneumonia detection system. The project follows a **decoupled client-server architecture**:

- **Frontend:** React 18 SPA built with Vite, running on port 3000
- **Backend:** Python Flask REST API running on port 5000
- **AI Model:** DenseNet121/EfficientNetB3 deep learning models (Keras)
- **Explainability:** Grad-CAM++ heatmap overlays
- **Chatbot:** Gemini AI integration for medical Q&A

Communication between frontend and backend happens via **HTTP REST calls** using the Fetch API. CORS is enabled on the Flask server to allow cross-origin requests from localhost:3000.

2. Technology Stack

Layer	Technology	Purpose
Frontend Framework	React 18.3.1	UI component rendering

Layer	Technology	Purpose
Build Tool	Vite 6.3.5	Dev server & bundling
Styling	Vanilla CSS	Glassmorphism dark theme
Fonts	Google Fonts (Inter, Outfit)	Typography
Backend	Flask (Python)	REST API server
CORS	flask-cors	Cross-origin requests
AI Model	TensorFlow/Keras	Pneumonia classification
Explainability	Grad-CAM++	Heatmap generation
Chatbot	Google Gemini API	Medical AI assistant

3. File Structure

```
RespiraCheck/
├─ frontend/
│   ├─ index.html           ← HTML entry point
│   ├─ package.json         ← Dependencies & scripts
│   ├─ vite.config.js       ← Vite dev server config
│   └─ src/
│       ├─ main.jsx         ← React DOM mount point
│       ├─ App.jsx          ← Main application component (440 lines)
│       └─ style.css         ← All CSS styles (750 lines)
├─ app.py                  ← Flask backend API (235 lines)
├─ gradcam.py              ← Grad-CAM++ module (280 lines)
├─ best_model_path.txt     ← Points to best trained model
├─ .env                   ← GEMINI_API_KEY (gitignored)
└─ *.keras                 ← Trained model weight files
```

4. Frontend Entry Point — index.html

Path: frontend/index.html

This is the single HTML page that Vite serves. Key responsibilities:

- Loads Google Fonts (Inter for body, Outfit for headings)
- Sets global box-sizing reset and dark background (#060a14)
- Prevents FOUC (Flash of Unstyled Content) with a fade-in animation on #root
- Mounts the React app into <div id="root">
- References src/main.jsx as the module entry point

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>RespiraCheck – AI Chest X-Ray Pneumonia Detection</title>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Outfit:wght@400;500;600;700;800&display=swap"
      rel="stylesheet" />
    <style>
      *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
      body { background: #060a14; font-family: 'Inter', sans-serif; }
      #root { opacity: 0; animation: rootFadeIn 0.4s 0.1s forwards; }
      @keyframes rootFadeIn { to { opacity: 1; } }
    </style>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

5. React Entry Point — main.jsx

Path: frontend/src/main.jsx

Minimal bootstrap file. Creates the React root and renders `<App />` inside `StrictMode`.

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

React.StrictMode enables additional development warnings for deprecated APIs and side-effect issues.

6. Vite Configuration — vite.config.js

Path: frontend/vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000,    // Matches Flask CORS expectations
    open: true,    // Auto-opens browser on npm run dev
  },
})
```

The `@vitejs/plugin-react` plugin enables JSX transformation and React Fast Refresh during development.

7. Main Application Component — App.jsx

Path: frontend/src/App.jsx (440 lines)

This is the core of the frontend. It is a single-file React component that handles the entire UI.

7.1 Imports & Constants

```
import React, { useState, useRef, useEffect } from "react";
import "./style.css";

const API_URL = "http://localhost:5000";
```

`API_URL` is the base URL for all backend API calls. The backend Flask server runs on port 5000.

7.2 SVG Icon Components (Lines 5-35)

Seven inline SVG icon components are defined as functional React components:

Component	Usage
<code>ActivityIcon</code>	Brand logo in navbar (heartbeat line)
<code>UploadCloudIcon</code>	Dropzone upload indicator
<code>XIcon</code>	Remove/close buttons
<code>ShieldAlertIcon</code>	Medical disclaimer icon

Component	Usage
ScanLineIcon	Grad-CAM section header
BotIcon	AI chatbot avatar
SendIcon	Chat send button
AlertCircleIcon	Error alert messages

7.3 Markdown Renderer (Lines 40-84)

```
const renderMarkdown = (text) => { ... }
```

A lightweight markdown parser for Gemini AI responses. It handles:

- Paragraph splitting (by double newline)
- Bullet lists (* or – prefixed lines)
- Bold text (**text** syntax)

This is used to render chatbot assistant messages with proper formatting.

7.4 State Management (Lines 87-99)

The App component uses React's `useState` hook for all state:

```
const [file, setFile] = useState(null); // Selected File object
const [preview, setPreview] = useState(null); // Object URL for preview
const [loading, setLoading] = useState(false); // Prediction in progress
const [result, setResult] = useState(null); // API response data
const [error, setError] = useState(null); // Error message string
const [isDragging, setIsDragging] = useState(false); // Drag-over visual state
const [chatHistory, setChatHistory] = useState([]); // Chat message array
const [chatInput, setChatInput] = useState(""); // Current chat input
const [isChatLoading, setIsChatLoading] = useState(false); // Chat request pending
```

Refs:

- `inputRef` — Hidden file input element reference
- `chatEndRef` — Invisible div at bottom of chat for auto-scroll

7.5 Side Effects (Lines 102-119)

Auto-scroll chat: Whenever `chatHistory` or `isChatLoading` changes, the chat panel scrolls to the bottom.

```
useEffect(() => {
  if (chatEndRef.current) {
    chatEndRef.current.scrollToView({ behavior: "smooth" });
  }
}, [chatHistory, isChatLoading]);
```

Initial bot message: When `result` is set (after a successful prediction), an initial assistant message is generated summarizing the prediction.

```
useEffect(() => {
  if (result) {
    const initialMsg = `I've analyzed the chest X-ray. The AI model predicts
    **${result.prediction}** with a confidence of
    **${(result.confidence * 100).toFixed(1)}%**...`;
    setChatHistory([{ role: "assistant", text: initialMsg }]);
  } else {
    setChatHistory([]);
  }
}, [result]);
```

7.6 File Handling (Lines 121-147)

handleFile(f): Validates file type (jpg/jpeg/png) and size (max 5MB), creates a preview URL via `URL.createObjectURL()`, and resets previous results.

handleClear(e): Resets all state — file, preview, result, error, and chat history.

7.7 Prediction Submission — handleSubmit (Lines 149-176)

This is the core integration point with the backend `/predict` endpoint:

```

const handleSubmit = async () => {
  if (!file) return;
  setLoading(true);
  setResult(null);
  setError(null);

  try {
    const formData = new FormData();
    formData.append("file", file);          // Attach the image file

    const res = await fetch(`${API_URL}/predict`, {
      method: "POST",
      body: formData,                      // multipart/form-data
    });

    const data = await res.json();

    if (!res.ok) {
      setError(data.error || "An unknown error occurred.");
    } else {
      setResult(data);
      // data = { prediction, confidence, gradcam_image }
    }
  } catch (err) {
    setError("Failed to connect to server. Is the backend running?");
  } finally {
    setLoading(false);
  }
};

```

Key integration details:

- Uses `FormData` to send the image as `multipart/form-data`
- The file is sent under the key `"file"` (must match `request.files["file"]` in Flask)
- Response JSON contains: `prediction` (string), `confidence` (float 0-1), `gradcam_image` (base64 PNG)

7.8 Chat Submission — `handleChatSubmit` (Lines 178-227)

Sends user messages to the backend `/chat` endpoint:

```
const handleChatSubmit = async (e) => {
  e.preventDefault();
  if (!chatInput.trim() || isChatLoading || !result) return;

  const userMsg = chatInput.trim();
  setChatInput("");

  // Add user message to UI immediately (optimistic update)
  const updatedHistory = [...chatHistory, { role: "user", text: userMsg }];
  setChatHistory(updatedHistory);
  setIsChatLoading(true);

  try {
    const res = await fetch(`${API_URL}/chat`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        message: userMsg,
        history: updatedHistory.slice(0, -1), // Previous messages
        prediction_context: {
          prediction: result.prediction,
          confidence: result.confidence
        }
      })
    });

    const data = await res.json();
    if (!res.ok) {
      setChatHistory(prev => [...prev, {
        role: "assistant",
        text: `Error: ${data.error}`
      }]);
    } else {
      setChatHistory(prev => [...prev, {
        role: "assistant",
        text: data.reply
      }]);
    }
  } catch (err) {
    setChatHistory(prev => [...prev, {
      role: "assistant",
      text: "Network error. Could not reach the chat server."
    }]);
  }
}
```



```
    } finally {  
        setIsChatLoading(false);  
    }  
};
```

Key integration details:

- Sends JSON body (not FormData)
- Includes full chat `history` array for context continuity
- Includes `prediction_context` so Gemini knows the current diagnosis
- Uses optimistic UI update — user message appears instantly before server responds

7.9 JSX Render Structure (Lines 232-438)

The render tree is organized as follows:

```

<>
<div class="bg-effects" />           ← Background gradient blobs
<div class="particles" />           ← Dot grid pattern overlay
<div class="app-container">
  <nav class="navbar">               ← Brand logo + name
  <main class="main-content">        ← Flexbox row layout
    <section class="scanner-section">
      <div class="glass-card">
        |— Card Header (title + subtitle)
        |— Error Alert (conditional)
        |— Dropzone OR Preview Area
        |— Analyze / New Image Button
        |— Results Section (conditional):
          |— Status Badge (NORMAL/PNEUMONIA)
          |— Confidence Bar (animated fill)
          |— Grad-CAM++ Image (base64)
          |— Medical Disclaimer
      </div>
    </section>

    <!-- Only visible after prediction -->
    <section class="chat-section">
      <div class="chat-panel">
        |— Chat Header (avatar + status)
        |— Chat Messages (scrollable)
        |   |— Message bubbles (user/assistant)
        |   |— Typing indicator (animated dots)
        |— Chat Input Area (form + send button)
      </div>
    </section>
  </main>
</div>
</>

```

Conditional rendering:

- The Dropzone is shown when no file is selected; replaced by Preview Area when a file exists
- The Results Section appears only after a successful prediction
- The Chat Section appears only when `result` is truthy (prediction complete)
- The Loading Overlay appears during prediction processing
- The scanner section shrinks from 800px to 600px max-width when results appear

8. Stylesheet — style.css

Path: frontend/src/style.css (750 lines)

8.1 CSS Custom Properties (Design Tokens)

```
:root {
  --bg-dark: #060a14;
  --bg-card: rgba(18, 25, 43, 0.6);
  --border-color: rgba(255, 255, 255, 0.08);
  --text-primary: #f0f4f8;
  --text-secondary: #94a3b8;
  --accent-blue: #3b82f6;
  --accent-purple: #8b5cf6;
  --accent-gradient: linear-gradient(135deg, var(--accent-blue), var(--accent-purple));
  --success: #10b981;
  --danger: #ef4444;
  --warning: #f59e0b;
  --font-heading: 'Outfit', sans-serif;
  --font-body: 'Inter', sans-serif;
  --shadow-glow: 0 0 40px rgba(59, 130, 246, 0.15);
  --shadow-card: 0 10px 30px rgba(0, 0, 0, 0.5);
}
```

8.2 Major CSS Sections

Section	Lines	Description
Background Effects	38-59	Radial gradient blobs + dot grid particles
Layout	61-69	Flex column container, max-width 1400px
Navbar	71-107	Gradient brand icon, gradient text fill
Main Content	109-132	Flexbox row, 32px gap, responsive transition
Glass Card	134-163	Glassmorphism: backdrop-filter blur, semi-transparent bg
Dropzone	165-209	Dashed border, hover/drag states, icon animation
Preview Area	211-264	Image display with gradient overlay for filename
Action Button	266-297	Gradient background, hover glow, disabled states
Loading Overlay	299-322	Full-cover blur with spinning border animation
Results	324-401	Status badges (green/red), animated confidence bar
Grad-CAM Viewer	403-423	Blue-bordered image display
Medical Disclaimer	425-446	Yellow-tinted warning box
Error Alert	448-460	Red-tinted box with shake animation
Chat Panel	462-679	Sticky sidebar, custom scrollbar, message bubbles
Animations	681-719	spin, fadeUp, slideDown, slideLeft, messageSlideIn, shake

Section	Lines	Description
Responsive	721-749	Breakpoints at 1024px and 640px

8.3 Key Design Techniques

Glassmorphism: Cards use `backdrop-filter: blur(16px)` with semi-transparent backgrounds.

Gradient Text: Brand name uses `background-clip: text` with `-webkit-text-fill-color: transparent`.

Animated Confidence Bar: Uses CSS `transition: width 1s cubic-bezier(0.4, 0, 0.2, 1)` for smooth fill.

Chat Typing Indicator: Three dots with staggered `animation-delay` using `typingBounce` keyframes.

Responsive Layout: At 1024px, the two-column layout stacks vertically; the chat panel becomes static instead of sticky.

9. Backend API — app.py

Path: `app.py` (235 lines)

9.1 Initialization

```
app = Flask(__name__)
CORS(app)  # Allows frontend on port 3000 to call port 5000
```

On startup (`__main__`):

- 1. Reads `best_model_path.txt` to find the best trained model
- 2. Loads the model using `safe_load_model()` from `gradcam.py`
- 3. Initializes Gemini AI client using `GEMINI_API_KEY` from `.env`
- 4. Starts Flask on `0.0.0.0:5000`

9.2 Endpoints

Method	Endpoint	Purpose	Request	Response
GET	<code>/health</code>	Health check	None	<code>{ status, model_loaded, model_path, gemini_available }</code>
POST	<code>/predict</code>	X-ray classification	multipart file	<code>{ prediction, confidence, gradcam_image }</code>
POST	<code>/chat</code>	AI chat	JSON body	<code>{ reply }</code>

9.3 `/predict` Endpoint Logic

- 1. Validates file exists and has allowed extension (`jpg/jpeg/png`)

2. Opens image with PIL, resizes to 224x224, normalizes pixels to [0,1]
3. Runs `model.predict()` — outputs sigmoid probability (0-1)
4. If probability > 0.5 → PNEUMONIA, else → NORMAL
5. Generates Grad-CAM++ heatmap via `generate_gradcam()`
6. Overlays heatmap on original image via `overlay_heatmap()`
7. Encodes overlay as base64 PNG string
8. Returns JSON: `{ prediction, confidence, gradcam_image }`

9.4 /chat Endpoint Logic

1. Validates JSON body contains "message"
 2. Extracts optional `prediction_context` and `history`
 3. Builds a full prompt with system instructions + context + conversation history
 4. Calls `gemini_client.models.generate_content()` with the prompt
 5. Returns `{ reply: response.text }`
-

10. Grad-CAM Module — gradcam.py

Path: `gradcam.py` (280 lines)

10.1 safe_load_model()

Monkey-patches Keras layer `from_config` methods to strip incompatible parameters (renorm, quantization_config, etc.) for cross-version compatibility.

10.2 generate_gradcam()

Implements Grad-CAM++ using triple-nested GradientTape:

1. Forward pass through model to get conv layer activations + predictions
2. Computes 1st, 2nd, and 3rd order gradients
3. Calculates alpha weights (Grad-CAM++ formula)
4. Produces weighted combination of feature maps
5. Applies ReLU and resizes to 224x224
6. Normalizes to [0,1] range

10.3 overlay_heatmap()

Applies JET colormap to heatmap and alpha-blends with original image using OpenCV.

11. Frontend-Backend Integration Flow

11.1 Image Upload & Prediction Flow

User Action	Frontend (React)	Backend (Flask)
1. Selects/drops image	→ handleFile() validates creates preview URL stores File object	
2. Clicks "Analyze"	→ handleSubmit() creates FormData appends file as "file" POST /predict (multipart/form-data) ← JSON response setResult(data) triggers chat useEffect	→ request.files["file"] _preprocess_image() model.predict() generate_gradcam() overlay_heatmap() ← base64 encode overlay return jsonify({...})
3. Results render	→ Status badge appears Confidence bar animates Grad-CAM image displays Chat panel slides in Initial bot message set	

11.2 Chat Interaction Flow

User Action	Frontend (React)	Backend (Flask)
1. Types message	→ <code>setChatInput(value)</code>	
2. Submits (Enter/btn)	→ <code>handleChatSubmit()</code> Optimistic UI update (user message appears) <code>POST /chat</code> { <code>message,</code> <code>history,</code> <code>prediction_context</code> } ← { <code>reply</code> } Append assistant msg Auto-scroll to bottom	→ Extract message Build system prompt + prediction context + conversation history Call Gemini API ← <code>return jsonify({reply})</code>

11.3 Key Integration Points Summary

Frontend Action	API Call	Backend Handler	Data Format
File upload + analyze	<code>POST /predict</code>	<code>predict()</code>	FormData → JSON response
Chat message	<code>POST /chat</code>	<code>chat()</code>	JSON → JSON
Health check	<code>GET /health</code>	<code>health()</code>	None → JSON

11.4 CORS Configuration

The Flask backend uses `flask-cors` with `CORS(app)` which allows ALL origins by default. The Vite dev server runs on port 3000, making cross-origin requests to port 5000.

11.5 Error Handling Chain

Frontend errors are handled at three levels:

- 1. **Client-side validation:** File type/size checked before upload
- 2. **HTTP error responses:** Backend returns error JSON with appropriate status codes (400, 413, 500, 503)
- 3. **Network failures:** Catch block handles connection refused / timeout scenarios

12. API Endpoint Reference

POST /predict

Request:

```
Content-Type: multipart/form-data
Body: file = <image.jpg>
```

Success Response (200):

```
{
  "prediction": "PNEUMONIA",
  "confidence": 0.9734,
  "gradcam_image": "iVBORw0KGgo..."
}
```

Error Responses:

- 400: Invalid file / no file / preprocessing failed
- 413: File exceeds 5MB
- 500: Model prediction failed
- 503: Model not loaded

POST /chat

Request:

```
{
  "message": "What does this result mean?",
  "history": [
    { "role": "assistant", "text": "I've analyzed..." },
    { "role": "user", "text": "Is this serious?" }
  ],
  "prediction_context": {
    "prediction": "PNEUMONIA",
    "confidence": 0.9734
  }
}
```

Success Response (200):

```
{
  "reply": "Based on the analysis, the AI model has detected..."
}
```

GET /health

Response (200):

```
{  
  "status": "ok",  
  "model_loaded": true,  
  "model_path": "best_densenet_phase2.keras",  
  "gemini_available": true  
}
```

13. How to Run

Start Backend:

```
cd RespiraCheck  
pip install -r requirements.txt  
python app.py  
# Server starts on http://localhost:5000
```

Start Frontend:

```
cd RespiraCheck/frontend  
npm install  
npm run dev  
# Opens http://localhost:3000
```

Both servers must be running simultaneously for the application to work.
